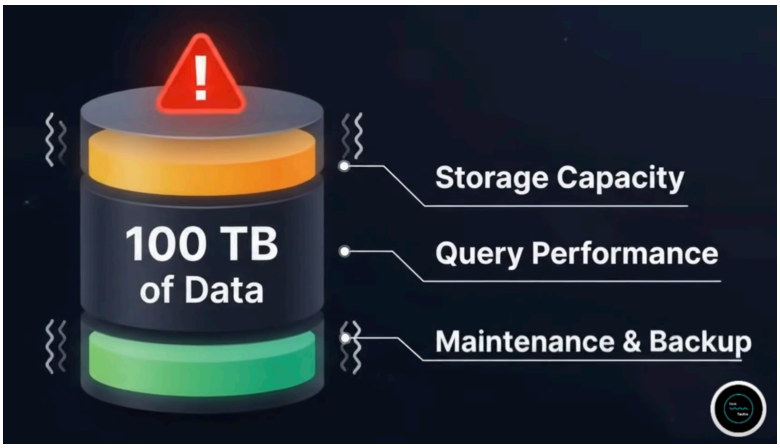
Database Replication & Sharding

DB today

DB Tomorrow



DATABASE REPLICATION vs SHARDING

Two different strategies for scaling your database

REPLICATION

Copy the same data to multiple databases

HOW IT WORKS

KEY BENEFITS

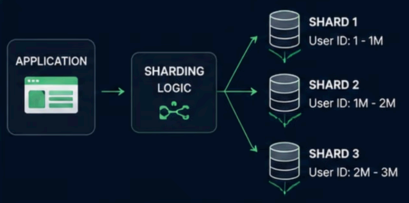
- High Availability (Failover)
- Read Scalability
- Data Redundancy
- Disaster Recovery

VS

SHARDING

Split data across multiple databases

HOW IT WORKS



KEY BENEFITS

- Horizontal Scalability (Large Data)
- Better Write Capacity
- Distributes Storage & Compute
- Handles Massive Datasets

Same data is copied to all replicas	DATA DISTRIBUTION	Different data is split across shards
All writes go to leader; replicas follow	WRITE HANDLING	Writes go to the appropriate shard
Helps with read scaling and high availability	PRIMARY GOAL	Helps with scaling large data and write throughput
Low (data is duplicated)	STORAGE REQUIREMENT	High Efficiency (no duplicate data)
Adding replicas is easy	SCALING COMPLEXITY	Resharding is complex (data movement)
Examples: MySQL Replication, PostgreSQL Streaming Replication, MongoDB Replica Set	COMMON EXAMPLES	Examples: MongoDB Sharding, Cassandra, DynamoDB, CockroachDB, Vitess

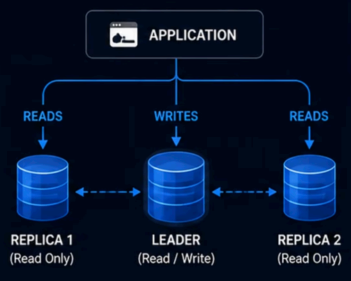
In real-world systems, Replication and Sharding are often used together: Sharding splits the data, Replication keeps each shard highly available.

Scaling Databases: Replication, Sharding & Both

Three ways to handle big traffic, big data, and high availability

1. REPLICATION

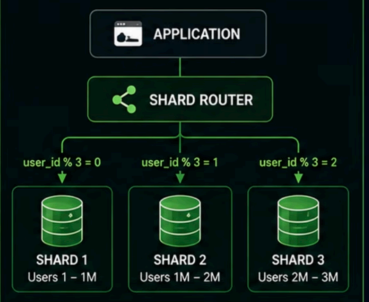
Scale database traffic & improve availability



Same Data, Multiple Copies
High Availability • Read Scaling

2. SHARDING

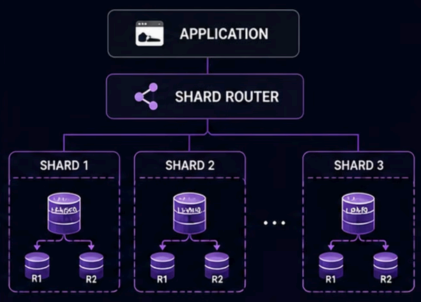
Distribute large dataset across multiple databases



Different Data, Different DBs
Horizontal Scaling • High Write Capacity

3. REPLICATION + SHARDING

Combine both for massive scale and best reliability



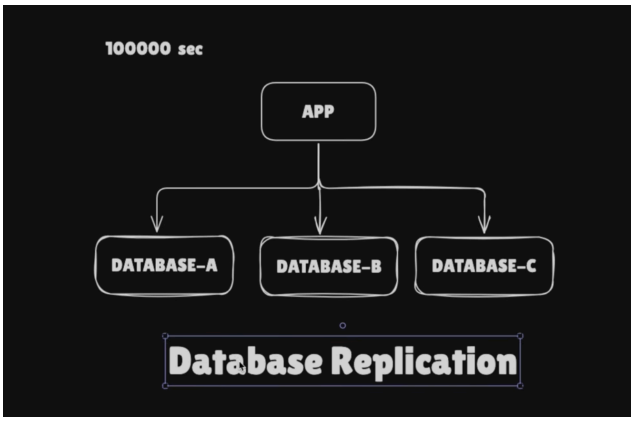
Split Data + Replicate Each Shard
Massive Scale • High Availability • High Performance

Replication = Copy
Same data on many nodes

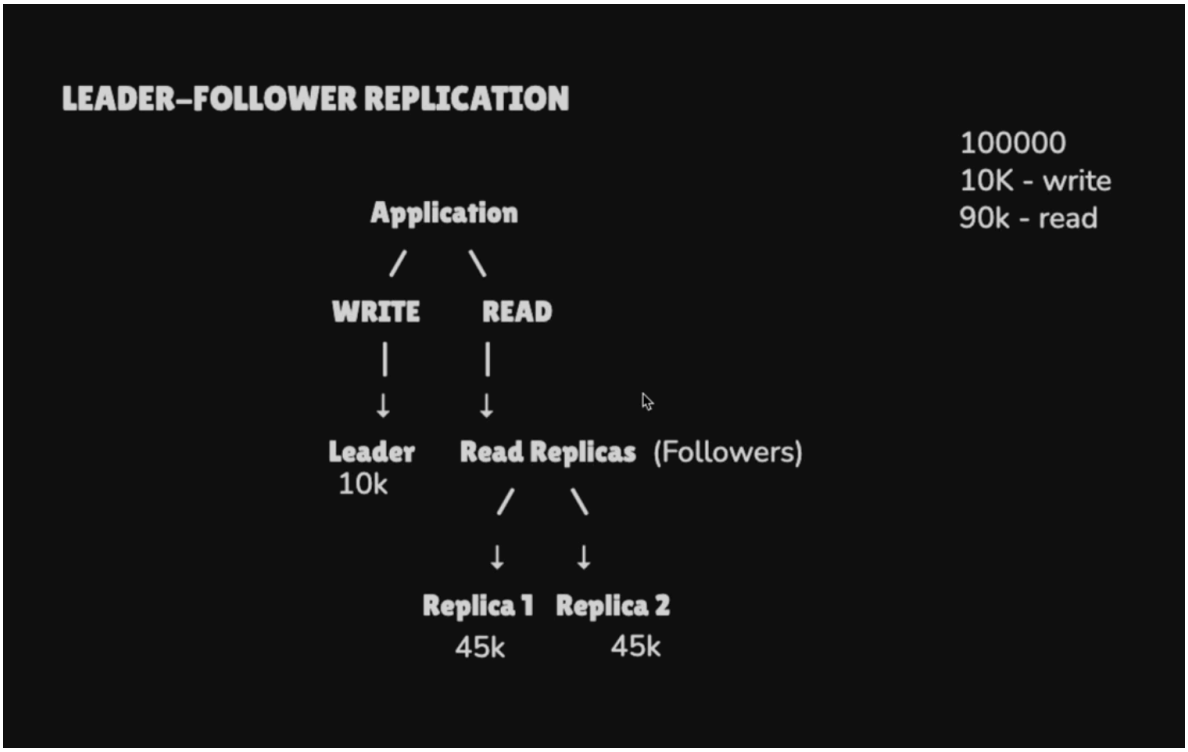
Sharding = Split
Different data on different nodes

Both Together = Best of Both
Scale without limits

Each DB contain copy

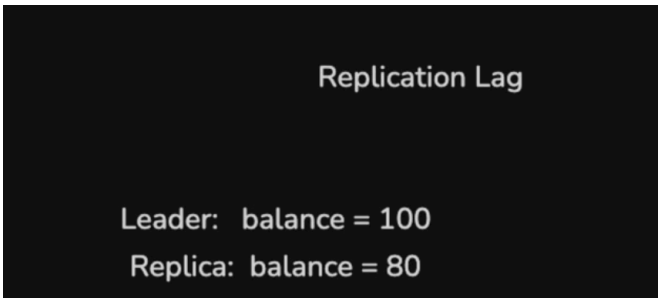


Who should handle write and read operations

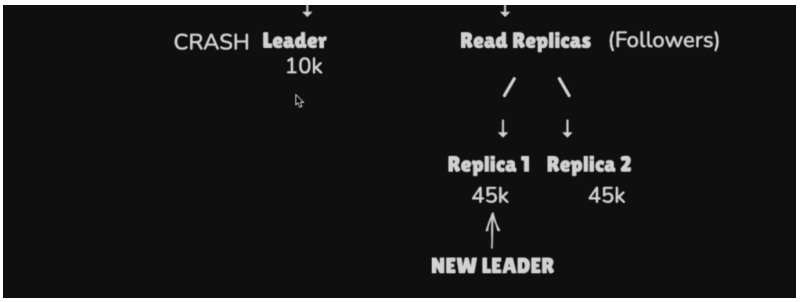


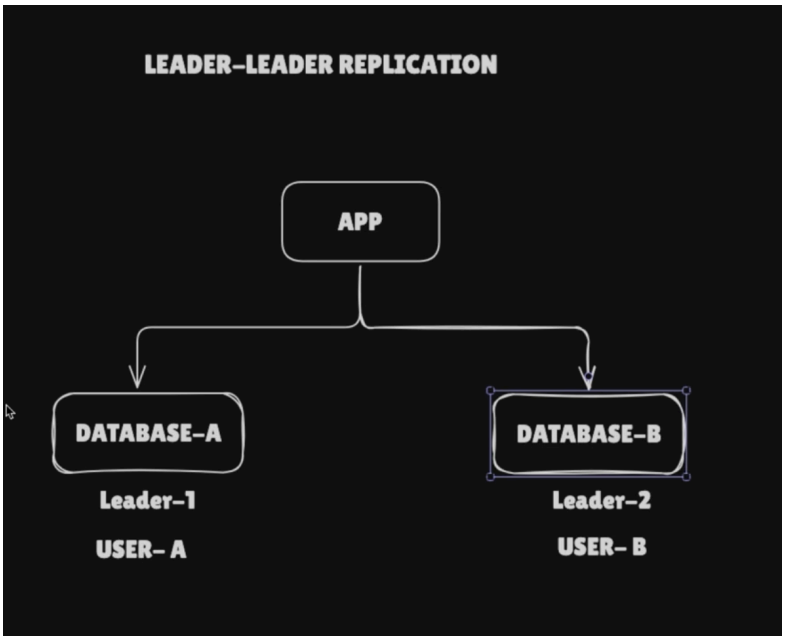
Synchronous update leader writes and wait for read replicas to acknowledge write. Latency may increase.

In Synchronous leader write and acknowledge to application write is completed. But asynchronous write will happen to replicas .in this case data in consistency will come in picture. Replica lag will be there.

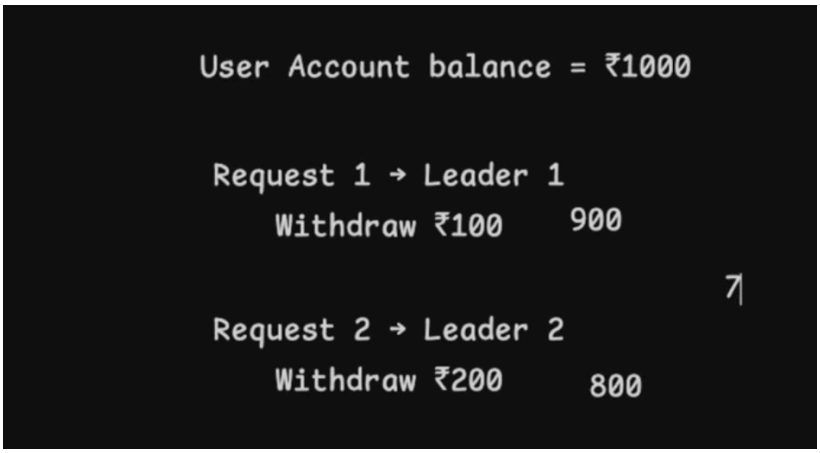


What if read itself crash. One of the replicas has to be promoted as new leader.

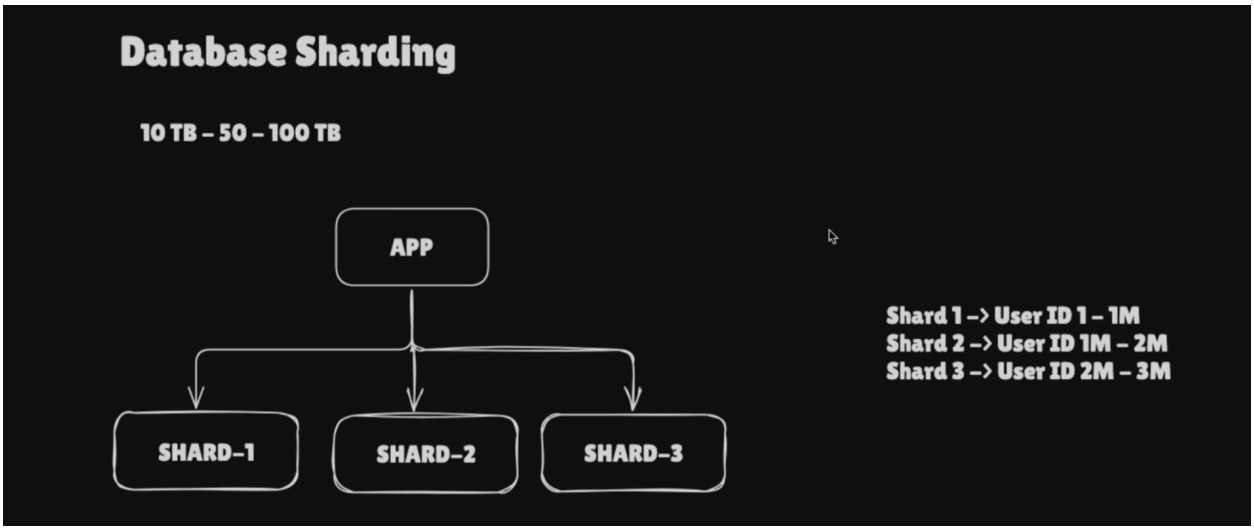


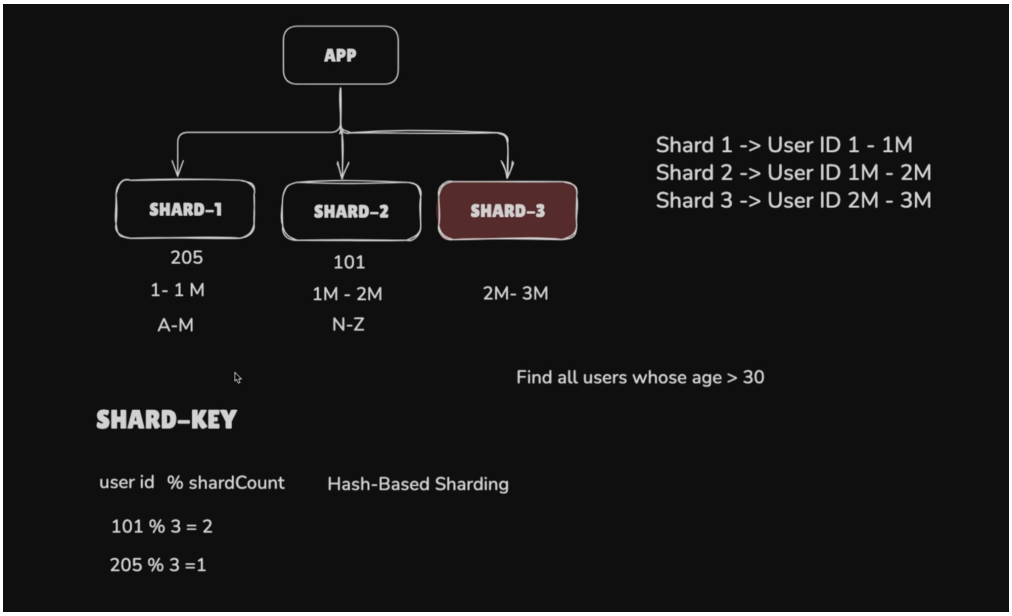


It overcomes from crash. But has write conflict when we have concurrent request.



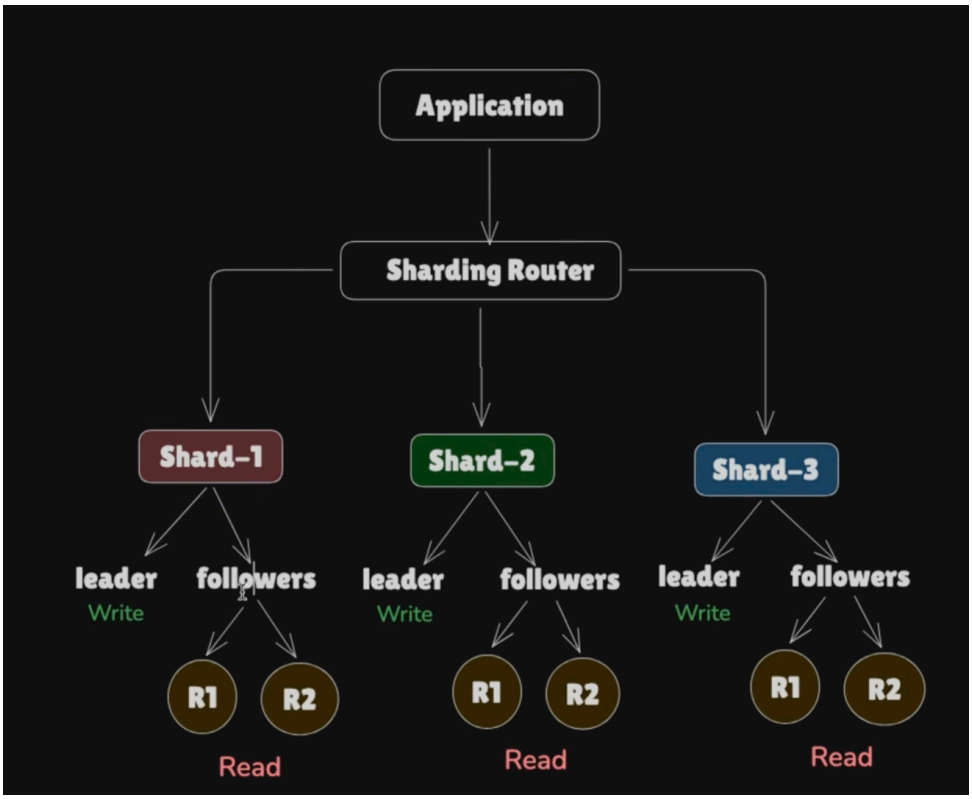
Database horizontal scaling –





Range-Based Sharding

Shard 1 -> User ID 1 - 1M
Shard 2 -> User ID 1M - 2M
Shard 3 -> User ID 2M - 3M



Sharding → Horizontal data and write scaling

Replication → Availability and read scaling